ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/365251029

Jenkins Pipelines: A Novel Approach to Machine Learning Operations (MLOps)

Conference Paper · October 2022

DOI: 10.1109/ICECAA55415.2022.9936252

CITATIONS
13

READS
320

2 authors:

Niranjan D R
R.V. College of Engineering
5 PUBLICATIONS **76** CITATIONS

SEE PROFILE

Mohana ..
R.V. College of Engineering
115 PUBLICATIONS **2,902** CITATIONS

SEE PROFILE

All content following this page was uploaded by Mohana .. on 14 November 2022.

The user has requested enhancement of the downloaded file.

Proceedings of the International Conference on Edge Computing and Applications (ICECAA 2022)
IEEE Xplore Part Number: CFP22BV8-ART; ISBN: 978-1-6654-8232-5

Jenkins Pipelines: A Novel Approach to Machine Learning Operations (MLOps)

Niranjan DR
Electronics and Telecommunication Engineering
RV College of Engineering®
Bengaluru, Karnataka, India

Mohana
Computer Science & Engineering (Cyber Security)
RV College of Engineering®
Bengaluru, Karnataka, India.

2022 International Conference on Edge Computing and Applications (ICECAA) | 978-1-6654-8232-5/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICECAA55415.2022.9936252

***Abstract* - Machine Learning is a widely popular field that is being used in an increasingly large number of projects worldwide. This necessitates the use of certain practices to create a structured framework for such projects. These practices, processes and pipelines are termed as Machine Learning Operations (MLOps). The software development lifecycle for a machine learning is indeed very complex and sequential nature of it results in several repetitive tasks for developers. Automation in this aspect would greatly reduce time and manual effort required. Jenkins is an open-source continuous integration tool that can be used to build pipelines to define and automate workflows in MLOps domain. This paper proposes the design and implementation of pipelines for various stages of MLOps that are data analysis, data preparation, training, testing and deployment on a single platform.**

Keywords—***Machine Learning Operations (MLOps), Software Development Lifecycle (SDLC), Machine Learning (ML), Continuous Integration (CI), Source Code Management (SCM).***

I. Introduction

The SDLC of a project defines certain practices which allow a project to be structured and easy to debug in case of errors or faults, even though there may be several stakeholders and developers working on it. SDLC has been a widely used framework for several decades now, and several SDLC standards have been set by regulatory authorities. However, since advent of Machine Learning in past few decades, several organizations and projects have started to implement Machine Learning to suit their use cases. When it comes to production, managing ML projects has been a difficult affair for several years now. During this time data scientists, analysts, developers and other stakeholders of an ML project have researched on ways to collaborate and work together. This collaboration has many uncertainties and complexities. Hence, it is not easy to overcome them. This is one of the major reasons why most ML projects do not reach the production phase at all. In recent past, several principles are being standardized, and tools being proposed to allow teams to collaborate on production-level ML projects. Development of these tools had multiple targets, preprocessing of data, model building, training the model, testing, deployment, and monitoring are some of them. As the field of AI progresses such kind of tools are constantly emerging.

MLOps involves several sequential processes such as data preprocessing, model training, model testing and model deployment. Throughout the lifecycle of an ML project, multiple changes are made to each of these stages. However, after the changes are made, need to build and run these stages from beginning, in a sequential order to makes it a repetitive and time-consuming process. Moreover, MLOps is a relatively new field, and standards are still being discussed and researched over. In this paper, Jenkins, a CI tool is used to automate various stages in MLOps. Jenkins uses pipelines which allow developers to define a structured workflow for sequential stages. A pipeline is set of jobs that are sequential or linked to one another. Using plugins, Jenkins implements CICD pipelines. Jenkins pipeline has several advantages. first of them being that the pipeline can easily be edited and changed by several users, which is especially useful in projects involving large teams. Secondly, even if system that the pipeline is running on loses power or switches off, the pipeline resumes the progress from where it left off. Most important of all, pipelines can be made to run parallel on multiple systems simultaneously, hence speeding up processes like model preprocessing and model training. Through this automation, if developer makes a change in the code for the data preprocessing stage, Jenkins CI would automatically build and run all the forthcoming stages without the need for manual intervention. This would greatly reduce time, manual effort and cost for ML projects. This method sequentially implements all the stages of the machine learning process, for changes made on a Source Code Management tool such as Git. An email notification is also sent to the engineer after completion of each stage, with information on the various results of the stage such as accuracy and error rate, and the error logs in case of faults in the program.

II. Literature survey

Yue Zhou *et al.* [1] discussed the processes of integration and deployment in ML systems and on how it differs from other software systems. Various tools were proposed to implement CICD pipeline. This included the likes of Kubeflow and ModelOps. Also, the consumption of resources, and time required were analysed for pipelines on each of these tools. This paper pioneered in the development of an end-to-end SDLC management for ML projects. P Karuppuswamy *et al.* [2] described the use of a random forest model for the industry use case of predictive maintenance. Predictive maintenance makes use of algorithms to predict failures of manufacturing machinery, to fix issues beforehand and to also optimise budgets. The paper discussed how the use of Machine Learning in this field increases the output and yield in manufacturing applications. The Random Forest algorithm was found to be more accurate than the Decision Tree algorithm. Arachchi *et al.* [3] identified that the approach of using Continuous

Authorized licensed use limited to: R V College of Engineering. Downloaded on November 09,2022 at 03:55:35 UTC from IEEE Xplore. Restrictions apply.

Integration and Continuous Delivery (CICD) pipeline increases the efficiency of large-scale projects. Timeline of a project and its delivery is of utmost importance to companies and to ensure that the client delivery date is met, a CICD pipeline becomes very vital. The test phase in CICD pipelines can sometimes not simulate the production performance behaviour as they cannot model the exact traffic in the real-world scenario. To solve this problem, the paper proposes a three-step automation phase with benchmark, load test and scaling. The paper was implemented using the Jenkins CI server. GoReplay was then used to simulate the traffic from the production environment point of view. Nikita Seth *et al.* [4] implemented Jenkins as a tool for software patch integration and releasing to client. The paper discusses, in detail, the concept of Continuous Integration (CI) and identifies that the software industry faces several challenges as they develop software versions at different sites. The problem becomes multifold as these various versions also need to be tested on different environments. Automation is identified in this paper as the best method of making this process more efficient. Considering a real-world scenario, the authors discuss how the Master-Slave architecture of Jenkins can be used along with several plugins to allow the automation tool to work with any given software development environment. Sriniketan Mysari *et al.* [5] focused on building and integrating Jenkins in collaboration with Ansible. Ansible is a configuration management tool which is widely used and is open source. The authors identified that several companies have multiple developers working on web projects and using Jenkins Ansible is an efficient and productive way of helping these developers collaborate with each other to create the product and make changes or updates. Ansible simplifies the Jenkins environment by making the SSH access process easier. The concept of using Jenkins for not only automating the build phase of the project but also the delivery phase. This multifaceted use of Jenkins allows all the stakeholders in the software development cycle, i.e., the Development team, Quality Assurance team and the Operations team can coordinate their efforts to have a quality product. The author discussed not only the merits of using Jenkins as a CD tool but also mentions the challenges that may be faced while doing so. Putu Adi Guna Permana *et al.* [6] identified the use of Jenkins Automation for Deployment along with the use of Scheduler and Notification. The authors identify that the key factors that companies aim for is a short delivery time and customer satisfaction. The different stages of software development are analysis, building or development, testing and deployment or delivery. Since these steps are manual, there are various problems that these corporations are facing. The authors mentioned an example where the developers who build the software must wait for several weeks or months to get test reports from the testing team. To solve this, the paper proposes Continuous Integration (CI) using Jenkins. The novelty of this paper is the fact that they also included ways of scheduling deployments and provides notifications to the various teams when the various stages of the automated process finish.

Heidilyn V. Gamido *et al.* [7] discussed and compared the features of various automated software testing tools. Software testing is an important part of the software development lifecycle as it verifies whether the user specifications and requirements are being met by the system or not. Testing can be either manual or automated. Automated testing has been proven to be more effective and time efficient. The paper compared several open source and commercial test automation tools to help users choose the best tool for their use case. Vinita Malik *et al.* [8] described automated software testing tools for web applications. The authors also emphasize that testing is a key component of the software development lifecycle to produce top quality applications. The paper compares several automated testing tools such as Selenium, Watir and Sahi. The architecture of all these testing tools was also well elaborated upon. The comparison was done based on use of resources in test script maintenance and increasing the efficiency when reusing the script. F Okezie *et al.* [9] discussed the important concept of software testing being cost and time efficient. The paper identified that the main reason why developers and organisations moved from manual testing to automated testing is because of the time and cost factors. Several comparisons are made between various automated testing tools, based on their platform of operation i.e., web applications, desktop, or mobile testing. The tools of different testing types such as functional testing, graphical user interface testing, unit testing, compatibility testing, regression testing, load testing and database testing are analysed through various trade-offs and the authors recommended the use of Test Complete and Ranorex for Desktop testing, Selenium for Web Application testing and Appium for mobile testing.

Prachi Paigude *et al.* [10] presented a test report automation process using Python. The use of APIs to connect between the automated testing tool and the Python script is notable here. The authors recognise that software testing is becoming increasingly complex as nowadays software's need to be tested on multiple operating environments. This takes a lot of time and manual effort, and the test report generation is ineffective and cumbersome. The process proposed in the paper automated the analysis of these test reports through a technique called text mining. Vector Cast tool was used for the generation of test reports in HTML format and a Python script was used to analyse these reports. Allan Mori *et al.* [11] addressed the problem of large set of test cases being generated as the testing activity progresses. Regression testing is a procedure in which any new update to the program or code is tested against the entire software or features of the software to ensure that the change does not affect any existing feature. Usually in manual regression testing, since the number of test cases is massive, the tester needs to choose the best test cases to make the testing process feasible with respect to time. The indications of whether a test case will allow the developer to assess the software on all terms, are termed as 'code smells'. The paper proposed a procedure to use anomaly detection to analyse code smells to help the user in the prioritisation of test cases. Emre Çeliket *et al.* [12] identified software testing as the sum of process that increase the product efficiency and reliability starting from the assessment of business requirements to the deployment of the product to the client or customer. The authors analysed that manual testing requires a human effort for thousands of different scenarios and it is also vulnerable to inaccurate results. Manual testing is also difficult to repeat and is very time consuming. The paper proves that automated testing improves the overall product quality and reduces vulnerabilities in testing. It also leads to faster version release times and less staff for manual testing in large corporations. Yi Zhao *et al.* [13] discussed that system testing is a growing practice in the software industry. The software industry is one

Authorized licensed use limited to: R V College of Engineering. Downloaded on November 09,2022 at 03:55:35 UTC from IEEE Xplore. Restrictions apply.

of the largest global industries and standardisation is of utmost importance to ensure less confusion and more collaboration. This paper assessed the possibility of standardisation in software system testing and tools used for it. Currently, each corporation uses a different testing methodology, pipeline, and software. The authors predicted that there will be standardisation of testing tools soon, in the industry in compliance with Quality Assurance and Control guidelines. Mitesh Soni *et al.* [14] described the use of Continuous Integration and Continuous Delivery (CICD) and DevOps in the insurance industry.

Mohammad Rizky Pratama *et al.* [15] explained performance test which a type of testing is used in the development phase to make sure that the software can accommodate the promised performance target and prevents bottlenecks due to large loads. The authors observe that the introduction of concepts like agile development and CI/CD have sped up the processes of build, test, and validation. This paper aimed at designing the CI/CD pipeline to automate the performance test and to reduce the human role in the process. Somya Garg *et al.* [16] described the use of CI/CD in cloud architecture and services. Cloud computing is a widely popular and fast-growing field. Almost all organisations are moving their databases and systems to the cloud. However, during this complex process of moving to the cloud, companies are facing difficulties in maintaining the structure of the software development pipeline. Docker is a service which uses virtualization at the operating system level to deploy software in packages which are commonly called containers. This paper used Docker to implement a CI/CD pipeline for effective maintenance and deployment with minimal or no manual effort. The role of continuous integration in a fast-growing information system industry. The entire industry is shifting towards a cloud architecture and since the business requirements of the customers or clients are increasingly complex and multiple, continuous integration delivery is vital for organisations to sustain their growth and maintain customer satisfaction. The authors also noted that, with the increase of such requirements, the tasks to be performed also become repetitive. Manual testing mechanisms are not fit for repetition as script reuse is infeasible. Therefore, automation of this phase using Jenkins along with Ansible. The Software Testing is a very important step in software development as it helps the developer assess whether the requirements are being met, to check errors or bugs and debug them to find solutions. Regression testing is a technique widely used in the software maintenance phase. It ensures that any small change in the software does not lead to the malfunction of any pre-existing feature. Test Case Prioritization and Optimization (TCPAO) techniques are used to save time and cost. This paper surveyed several techniques to select the best test cases and compared them.

Ratha Tim *et al.* [17] implemented Continuous Integration processes for client-side web applications. Though CI is easy to implement on software systems, it is difficult to do so on web applications. Even if the integration process is done manually, errors are usually found while integrating and testing the web application. This is because testing in web applications is different as the developer needs to test the features by clicking on each of them and run the application on different web browsers. Hence this paper demonstrated a new CI approach to better approach repeating processes and testing in web applications. Several organisations are increasingly using DevOps practices in which developers must work with several other departments of the organisation such as Quality Assurance and Control and the Business teams to ensure timely delivery and maintenance of the project. This paper presented an automated pipeline on Amazon Web Services. It used Jenkins for the CI stage, Ansible for CD and Kubernetes to deploy the application on to AWS. The design of a CI/CD system for programming code. Languages such as JAVA, MySQL, Gradle, Maven and REST were used. The application build was done using a 3-tier architecture to have better scalability, speed, availability, reliability, and serviceability. The novelty of this paper was that it proposed a design in which the Continuous Development phase is well distributed among resources to consider all the business considerations from the client's perspective. This paper too used the Agile development model.

From the literature survey, it was inferred that automated testing has several advantages over manual software testing through various analyses and examples given in literature. Jenkins is one of the most widely mentioned tool in literature and the advantages mentioned are that it is open source, easy to install, multiple OS compatible, diverse plugin library and a very strong community for support. The literature lacked research on connecting testing software's with automation tools such as Jenkins which is extremely important to automate the system testing processes in the CI/CD pipeline of organisations.

III. Design and Implementation Methodology

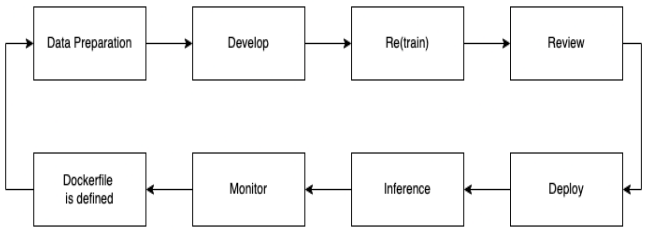

Fig.1. MLOps Cycle

MLOps is a cyclic process. Figure 1 illustrates the major steps in this process. Exploratory Data Analysis or EDA is the step in which the problem statement is analyzed, and the data is explored for the ML lifecycle through datasets and visualizations. Data preparation involves the transformation, aggregation and sometimes omission of unnecessary data through methods such as Principal Component Analysis (PCA) and Feature Subset Selection to make sure that the data is fit to be used to train the model.

Then the model is trained and tuned with the help of several ML libraries such as scikit-learn and optuna to improve the hyperparameters. Next, the model reviewed by analyzing the history of changes to the model and the several versions of the model. Model inference includes the process of analyzing the results from the model and measuring the accuracy to verify that the model can be deployed onto the production level environment. Then the model is deployed onto a server or cloud-based environment through applications such as Docker. This process repeats and the model is retrained iteratively until the desired results are achieved. Methodology and implementation as shown in figure 2.

Authorized licensed use limited to: R V College of Engineering. Downloaded on November 09,2022 at 03:55:35 UTC from IEEE Xplore. Restrictions apply.

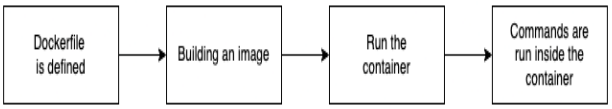

Fig.2. Methodology of Implementation

- The Dockerfile is defined with all the necessary prerequisites required for the pipeline. It is a contains all the commands that aid in assembling the image, later. Here, it contains information on where to run the pipeline. In this case, Jupyter Notebooks is chosen as the environment.
- Once the Dockerfile is defined an image is built for the machine learning model
- A container contains everything necessary to run the application including dependencies and environment variables. In this step, a container is built based on the image created earlier and is run.
- Now various commands on the script can be run easily in these containers. This allows the setting up Jenkins, later, to automate the various stages in an ML project.

It must be noted that this paper uses Docker, because it is a good practice in software development to use containerized applications. This is done to ensure that the application works similarly when run on any machine in the production environment since the environment variables and dependencies required by the application are present in the container.

The process of creation of Dockerfile is quite simple. The Dockerfile can be seen as a raw text file. It contains the following instructions:

- FROM – This is the first entry in any Dockerfile and defines the parent image.
- ARG – Once the Docker image is run, the variables that are defined in this section are used.
- CMD – Upon the creation of the Docker container this command is executed. Only one CMD instruction can be used per Dockerfile
- LABEL – It includes information about the docker file in key-value pairs such as the author's name and version details.
- RUN – These are the instructions to be executed when the image is built.
- USER – This defines the username of the user that created the Dockerfile.
- ADD – This is used to move files from a source to the required destination.
- EXPOSE – During runtime, Docker listens to the port mentioned in this instruction.
- ENV – Used to set the environment variables so that they are similar on both the source system and the production environment.
- WORKDIR – The current directory on which the Dockerfile must be executed

IV. DESIGN AND IMPLEMENTATION

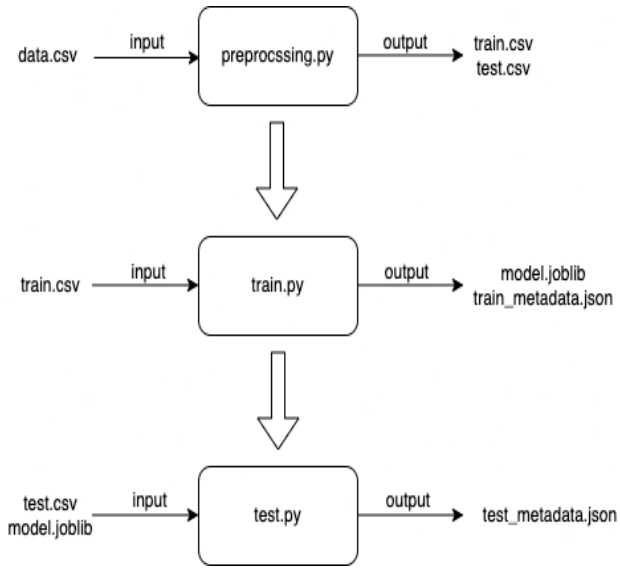

Fig.3. Block diagram of Implementation.

Figure 3 illustrates the block diagram implementation. For this example, the real estate price prediction is chosen. The paper considers three different containers to execute preprocessing.py, train.py and test.py. The preprocessing.py file receives the CSV file of the data as input and splits it into the training and test sets accordingly. The train and test CSV files are then passed on to the next stages as input. The train.py file then uses the training data set to train the model and outputs the trained model's Joblib file and the meta data from the logs of the training.

The test.py file receives the test data and the Joblib file as input and stores the output in a test metadata file. These three stages are implemented on Jenkins to automate the entire process of ML projects. Several times, there are changes that are required to be made in the dataset. In that case, the user can use the Jenkins pipeline to automatically go through all these three stages immediately after the change in the data.csv file is committed on Git or a similar SCM tool. If a change is made in the preprocessing, training or the testing phases, the next stages can be automatically run, accordingly. Since ML projects usually involve several changes to both the data and the model parameters, several times throughout the project, to obtain better accuracies, this process of automating the stages uses Jenkins saves time and effort. The software requirements of this implementation include a minimum of 1GB of RAM and 10GB of Storage for the Docker container to run. It also requires the installation of the latest version of JDK (Java Development Kit), and a web browser such as Google Chrome. It can be implemented on major OS' such as Windows, macOS and Linux.

Authorized licensed use limited to: R V College of Engineering. Downloaded on November 09,2022 at 03:55:35 UTC from IEEE Xplore. Restrictions apply.

Fig.4. Pipeline Script on Jenkins

Figure 4 shows the pipeline script on Jenkins. The pipeline has one stage for preprocessing, training and testing. Each stage has a script to execute a shell command. The shell commands that are executed for each of the stages are illustrated in Figure 5.

```
Preprocessing Stage :

sudo -S docker container exec model python3 preprocessing.py

Training Stage :

sudo -S docker container exec model python3 train.py

Testing Stage :

val_acc=$(sudo -S docker container exec model  jq .validation_acc \
/home/niranjandr/results/train_metadata.json)
threshold=0.8

if echo "$threshold > $val_acc" | bc -l | grep -q 1
then
        echo 'validation accuracy is lower than the threshold, process stopped'
else
  echo 'validation accuracy is higher than the threshold'
  sudo -S docker container exec model python3 test.py
  sudo -S docker container exec model cat \ /home/niranjandr/results/train_metadata.json \
/home/niranjandr/results/test_metadata.json
fi

sudo -S docker rm -f model
```

Fig.5. Shell commands required for different stages of the pipeline

The preprocessing stage contains the command which runs the container for the python script which splits the dataset into the training and testing data. Then the training stage container is used to train the model.

To add versatility to the automation, the testing stage contains a validation accuracy threshold which stops the testing process when the validation accuracy is lower than the set threshold.

VI. RESULTS AND ANALYSIS

Fig.6. Successful Pipeline run

The Jenkins Pipeline was successfully able to run through the three major stages of the project and helped automate the redundant tasks after each change in the data or the source code. For an ML project without this automation, the user would have to check the status of each stage before running the next. The entire process of monitoring and executing will have to be done after any change. However, this implementation automatically runs the stages once a change is made on the SCM tool.
After the completion of the stages, or during any error in one of the stages, the error logs or success report is sent via Mail, with the help of the SMTP plugin in Jenkins.

The model accuracy was found to be 97.3% which is on par with the standard accuracy on this dataset. This shows that the automated MLOps pipeline was able to achieve the accuracy generated by manual tweaking of the model, in much lesser time and with none or minimal manual effort.

VII. IMPLEMENTATION CHALLENGES IN MLOPS

Though MLOps is a promising field, it does pose several challenges. Firstly, unlike other software projects, a small change in the data almost always has a major effect on the output of the model. Hence any ML project is an iterative process. This requires very quick response times, from code integration to deployment.

Also, there are a multitude of tools and people involved in every ML project. This includes analysts, business executives and IT personnel. The pipeline, therefore, must be highly customizable, and easy to handle.
The lack of implementation of SDLC standards in MLOps is another major hurdle. MLOps is a novel field and hence lacks sufficient research and literature. The implementation presented in this paper proposes to solve all the challenges.
The limitations of existing methods for MLOps are that they are not completely automated or only a few steps such as preprocessing are partially automated. This leads to several redundant tasks being done manually especially in the training and testing stages of the ML project.

VIII. CONCLUSION AND FUTURE SCOPE

A. *Conclusion*

From the obtained results it can be observed that the automation of three major stages of ML project reduces manual effort required to run training and testing stages, every time a change is done to the data or preprocessing steps. Since industries and ML is moving towards data driven algorithms and technologies, this type of sequential automation will reduce manual time by automating most of the redundant tasks in the process of ML. The same Jenkins pipeline can be used for several projects with very few modifications. This leads to all projects in an organization to come under one umbrella, making it easier for engineers, analysts and business executives to better understand the project and work in collaboration. Use of an automated Jenkins pipeline is a more hassle-free and faster experience for ML projects than manual monitoring and execution. This is extremely useful in cases of teams working on projects, where a stable change made on SCM tool can immediately run stages and deploy updated model to cloud services for the end-user to access immediately. This implementation also greatly reduces time between code base changes and deployment of the project.

Authorized licensed use limited to: R V College of Engineering. Downloaded on November 09,2022 at 03:55:35 UTC from IEEE Xplore. Restrictions apply.

B. Future scope

In case of an organization using multiple systems to run and train such machine learning models, the Jenkins parallel function can be used to run the pipeline for multiple agents (systems) at once. This would reduce the time required for training ML models, by multifold [18] [19] [20]. In the long run, it can be expected that most of organizations will implement MLOps strategies which lead to more efficiency, accuracy and productivity. MLOps also allows ML to be implemented in various kinds of industries such as healthcare, aerospace and hospitality while solving the complexities that are associated with ML projects [21] [22] [23].

REFERENCES

[1] Y. Zhou *et al.* "Towards MLOps: A Case Study of ML Pipeline Platform," *International Conference on Artificial Intelligence and Computer Engineering (ICAICE),* 2020.

[2] P Karuppusamy *et al.* "Machine Learning Approach to Predictive Maintenance in Manufacturing Industry-A Comparative Study", *Journal of Soft Computing Paradigm vol.*2, pp.246-255, 2021.

[3] S. A. I. B. S. Arachchi *et al.* "Continuous Integration and Continuous Delivery Pipeline Automation for Agile Software Project Management", *Moratuwa Engineering Research Conference (MERCon),* 2018.

[4] N. Seth *et al.* "ACI (automated Continuous Integration) using Jenkins: Key for successful embedded Software development", *International Conference on Recent Advances in Engineering & Computational Sciences (RAECS)*, 2015.

[5] S. Mysari *et al.* "Continuous Integration and Continuous Deployment Pipeline Automation Using Jenkins Ansible", *International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE),* 2020.

[6] P. A. G. Permana *et al.* "Implementation of Jenkins Automation Deployment with Scheduler and Notification", *3rd International Conference on Cybernetics and Intelligent System (ICORIS),* 2021.

[7] Heidilyn V. *et al.* "Comparative review of the features of automated software testing tools", *International Journal of Electrical and Computer Engineering (IJECE),*Vol. 9, No.5*,* 2019.

[8] Vinita Malik *et al.* "Comparative Study of Automated Web Testing Tools" *International Journal of Latest Trends in Engineering and Technology (IJLTET),* 2016.

[9] F. Okezie *et al.* "A Critical Analysis of Software Testing Tools"*, International Conference on Engineering for Sustainable World*, 2019.

[10] P. Paigude *et al.* "Software Integration Test Report Analysis Automation Using Python", *Asian Conference on Innovation in Technology (ASIANCON),* 2021.

[11] A. Mori *et al.* "Anomaly Analyses to Guide Software Testing Activity", *IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 2020.

[12] E. Çelik *et al.* "Software test automation and a sample practice for an enterprise business software," *International Conference on Computer Science and Engineering (UBMK),* 2017.

[13] Y. Zhao *et al.* "Research on International Standardization of Software Quality and Software Testing", *IEEE/ACIS 20th International Fall Conference on Computer and Information Science (ICIS),* 2021.

[14] M. Soni *et al.* "End to End Automation on Cloud with Build Pipeline: The Case for DevOps in Insurance Industry, Continuous Integration, Continuous Testing, and Continuous Delivery", *IEEE International Conference on Cloud Computing in Emerging Markets (CCEM),* 2015.

[15] M. R. Pratama *et al.* "Implementation of Continuous Integration and Continuous Delivery (CI/CD) on Automatic Performance Testing", *9th International Conference on Information and Communication Technology (ICoICT),* 2021.

[16] S. Garg *et al.* "Automated Cloud Infrastructure, Continuous Integration and Continuous Delivery using Docker with Robust Container Security", *IEEE Conference on Multimedia Information Processing and Retrieval (MIPR),* 2019.

[17] R. Tim *et al.* "Continuous integration processes for modern client-side web applications", *International Electrical Engineering Congress (iEECON),* 2017.

[18] A Biswas *et al.* "Survey on Edge Computing–Key Technology in Retail Industry" *Lecture Notes on Data Engineering and Communications Technologies, vol 58*. Springer.

[19] A. Biswas *et al.* "Classification of Objects in Video Records using Neural Network Framework," *International Conference on Smart Systems and Inventive Technology (ICSSIT),* 2018, pp. 564-569.

[20] C. Kumar B *et al.* "Performance Analysis of Object Detection Algorithm for Intelligent Traffic Surveillance System," *International Conference on Inventive Research in Computing Applications (ICIRCA),* 2020, pp. 573-579.

[21] R. J. Franklin *et al.* "Traffic Signal Violation Detection using Artificial Intelligence and Deep Learning," *2020 5th International Conference on Communication and Electronics Systems (ICCES), 2020*, pp. 839-844.

[22] V. Rajesh *et al.*"Quantum Convolutional Neural Networks (QCNN) Using Deep Learning for Computer Vision Applications," *2021 International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT), 2021*, pp. 728-734.

[23] E. Shreyas *et al*. "3D Object Detection and Tracking Methods using Deep Learning for Computer Vision Applications," *2021 International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT), 2021*, pp. 735-738.

Authorized licensed use limited to: R V College of Engineering. Downloaded on November 09,2022 at 03:55:35 UTC from IEEE Xplore. Restrictions apply.

Publication Link:

https://ieeexplore.ieee.org/document/9936252

Cite This:

Niranjan DR and Mohana, **"Jenkins Pipelines: A Novel Approach to Machine Learning Operations (MLOps),"** 2022 International Conference on Edge Computing and Applications (ICECAA), 2022, pp. 1292-1297, doi: 10.1109/ICECAA55415.2022.9936252.

View publication stats